

STUDENT PERFORMANCE ANALYSER

Submitted by :- Avika Singh

Registration Number :-26BAI10458

Course:- BTech in CSE(AI/ML)

Subject:-Introduction to problem Solving and Programming

Course code:-CSE1021

Faculty:- Dr. Siddharth Singh Chouhan

1. INTRODUCTION

The Smart Student Performance Analyser is a Python-based project developed to analyse the academic performance of multiple students.

The program allows the user to enter student details, registration number, subjects and marks. It then calculates the total marks, average, percentage, grade, failed subjects, highest marks, lowest marks, and overall Pass/Fail result.

The project uses basic Python concepts such as functions, dictionaries, lists, loops, conditional statements, input/output, and modular programming. The main purpose of this project is to apply Python programming concepts to a practical problem related to student performance analysis.

2. PROBLEM STATEMENT

Managing and analysing the academic performance of multiple students manually can take time and may lead to calculation errors. It can be difficult to quickly calculate total marks, average, percentage, grades, failed subjects, and overall results for each student.

The Smart Student Performance Analyser aims to solve this problem by providing a simple Python-based system that takes student marks as input and automatically performs the required calculations. This makes the process more organised, faster, and easier to understand.

3. OBJECTIVES

The main objectives of the Smart Student Performance Analyser are:

- To develop a simple Python program for analysing student performance.
- To store and manage details of multiple students.
- To calculate total marks, average, and percentage automatically.
- To identify the highest and lowest marks obtained by a student.
- To identify subjects in which a student has failed.
- To assign grades based on the percentage obtained.
- To determine the overall Pass/Fail result.

To apply Python programming concepts such as functions, loops, dictionaries, lists, and conditional statements in a practical project.

4. SCOPE OF THE PROJECT

The scope of the Smart Student Performance Analyser includes:

- Entering details of multiple students.
- Entering the subjects and marks obtained by each student.
- Calculating total marks and average marks.
- Calculating the percentage of each student.
- Finding the highest and lowest marks.
- Identifying failed subjects.
- Assigning grades according to the percentage.
- Displaying the overall Pass/Fail result.
- Validating marks to prevent negative values.

The current project is designed as a basic command-line Python application. Future versions can include features such as a graphical user interface, data storage, report generation, and graphical representation of student performance.

5. TARGET USERS

The Smart Student Performance Analyser is mainly designed for:

- Students – to check and understand their academic performance.
- Teachers/Faculty – to analyse the performance of multiple students.
- Educational Institutions – as a basic tool for managing and analysing student results.

The project is simple to use and can be useful wherever basic student marks analysis is required.

6. FUNCTIONAL REQUIREMENTS

The Smart Student Performance Analyser should provide the following main functions:

1. Student Details Input

The system should accept the student's name and registration number.

2. Subject and Marks Input

The system should allow the user to enter the subjects and marks obtained by each student.

3. Performance Calculation

The system should calculate total marks, average marks, and percentage.

4. Grade Calculation

The system should assign a grade based on the student's percentage.

5. Pass/Fail Analysis

The system should identify failed subjects and display the overall Pass/Fail result.

6. Highest and Lowest Marks

The system should find and display the subjects with the highest and lowest marks.

7. Multiple Student Support

The system should allow performance details to be entered and analysed for multiple students.

8. Input Validation:- Checks if input is valid

7. NON-FUNCTIONAL REQUIREMENTS

The Smart Student Performance Analyser should meet the following non-functional requirements:

1. Usability

The program should be simple and easy to use through clear input and output messages.

2. Reliability

The program should provide consistent results for valid inputs.

3. Maintainability

The code should be divided into separate modules and functions so that it can be easily understood and modified.

4. Performance

The program should perform calculations and display results quickly without unnecessary processing.

5. Error Handling

The program should handle invalid marks, such as negative values, and ask the user to enter valid marks again.

6. Portability

The program should be able to run on systems that have Python installed.

8. TECHNOLOGIES AND TOOLS USED

The following technologies and tools were used to develop the Smart Student Performance Analyser:

Programming Language: Python

Code Editor: Visual Studio Code (VS Code)

Version Control: Git and GitHub

Libraries: Python's built-in features and functions

Data Handling: Lists and dictionaries

Programming Concepts: Functions, loops, conditional statements, input/output, and modular programming

9. PROJECT STRUCTURE

The project is divided into separate Python files to keep the code organised and easy to understand.

Smart Student Performance Analyser

```
├── main.py
├── student.py
├── analysis.py
├── validation.py
├── README.md
└── requirements.txt
```

Description of Files

- main.py – Controls the main flow of the program and displays the final performance report.
- student.py – Takes student details, subjects, and marks as input and supports multiple students.
- analysis.py – Performs calculations such as total, average, percentage, grade, highest/lowest marks, and failed subjects.
- validation.py – Checks whether the entered marks are valid.
- README.md – Contains information about the project, features, setup, and usage.
- requirements.txt – Contains the project requirements.

10. SYSTEM ARCHITECTURE

The project follows a simple modular architecture where different tasks are divided into separate Python files.

Main Components

1. User Input Module – student.py

Takes the student's name, registration number, subjects, and marks.

2. Validation Module – validation.py

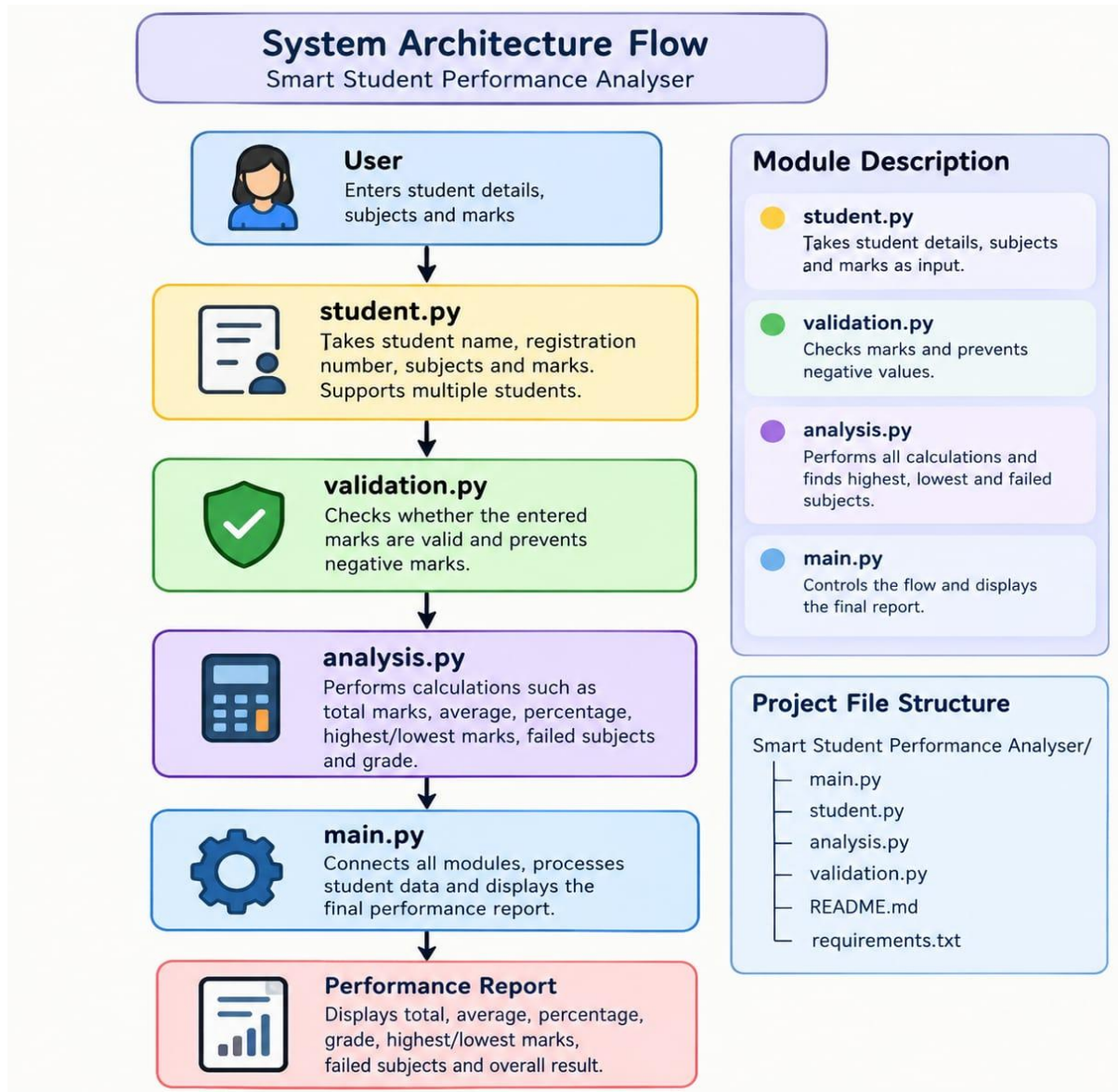
Checks whether the entered marks are valid and prevents negative marks.

3. Analysis Module – analysis.py

Performs calculations such as total marks, average, percentage, highest marks, lowest marks, and failed subjects.

4. Main Control Module – main.py

Connects the different modules, processes the student data, and displays the final performance report.



This modular structure makes the project easier to understand, maintain, and modify.

11. PROGRAM WORKFLOW

The program follows a simple step-by-step workflow:

1. The user enters the number of students.
2. The program takes each student's name and registration number.
3. The user enters the number of subjects and the marks obtained in each subject.
4. The validation module checks that the entered marks are not negative.
5. The program calculates the total marks, average, and percentage.
6. It identifies the highest and lowest marks.
7. It checks for failed subjects based on the passing criteria.
8. A grade is assigned according to the percentage.
9. The program determines the overall Pass/Fail result.
10. The final performance details are displayed for each student.

12. DESIGN DECISIONS

The following design decisions were made while developing the Smart Student Performance Analyser:

Modular Programming: The project is divided into separate files such as `student.py`, `analysis.py`, and `validation.py` to keep the code organised.

Functions: Functions are used for different tasks such as calculating marks, finding grades, and checking the final result.

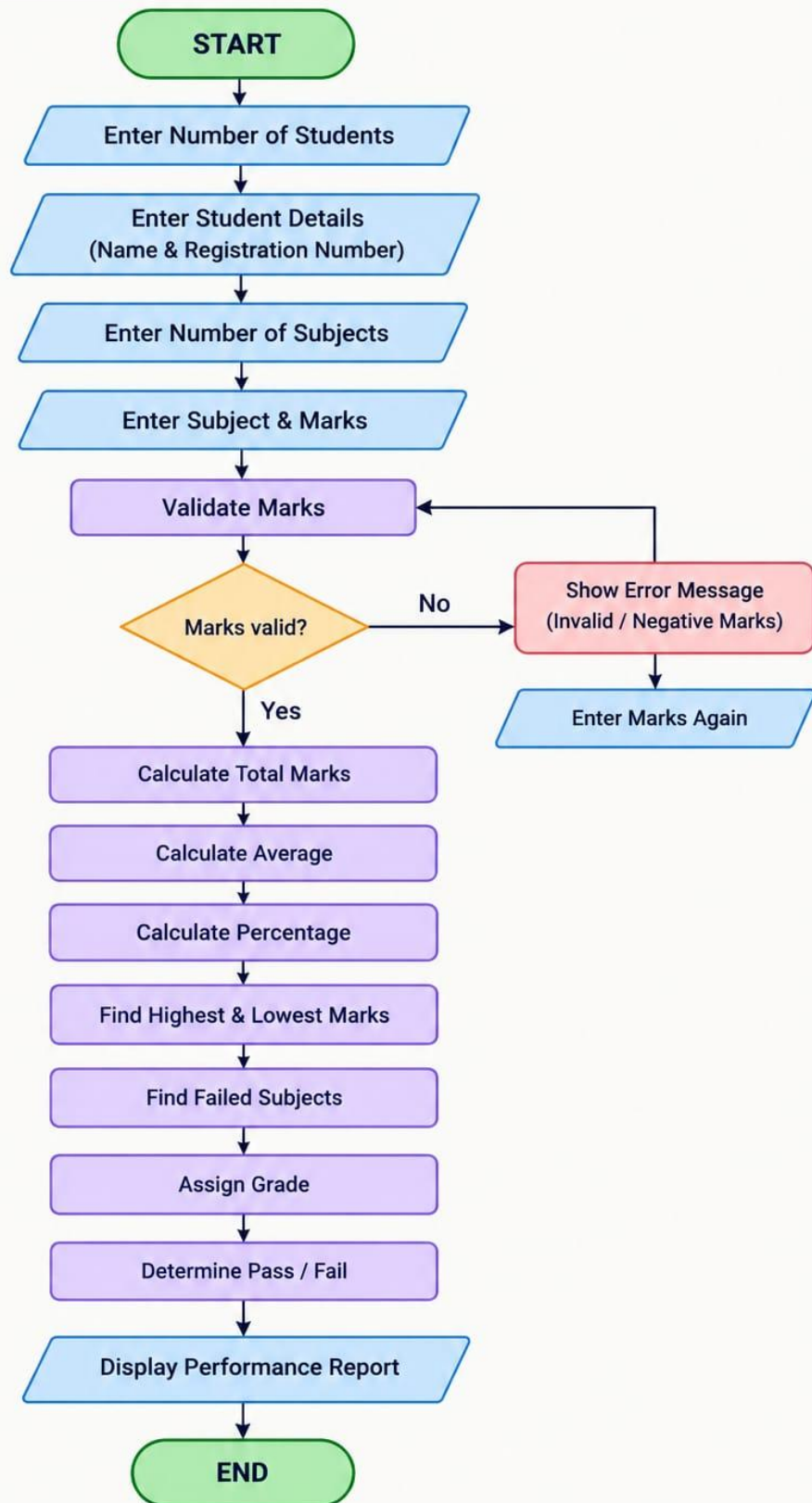
Lists and Dictionaries: Dictionaries are used to store student details and subject-wise marks, while lists are used for storing multiple students and failed subjects.

Simple User Input: The program uses a command-line interface so that it remains simple and easy to operate.

Input Validation: A separate validation function is used to prevent negative marks from being entered.

No Database: A database was not used because the current version focuses on basic Python programming concepts and does not require permanent data storage.

Program Workflow



13. IMPLEMENTATION

The Smart Student Performance Analyser was implemented using Python and divided into different modules. Each module performs a specific task.

13.1 Student Input

The student.py module collects the student's name, registration number, subjects, and marks. It also allows details of multiple students to be entered.

13.2 Input Validation

The validation.py module checks whether the entered marks are valid. Negative marks are rejected and the user is asked to enter the marks again.

13.3 Performance Analysis

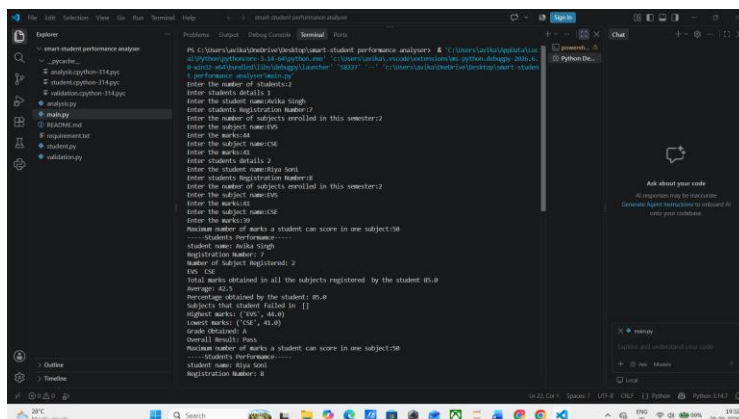
The analysis.py module performs the main calculations, including:

- Total marks
- Average marks
- Percentage
- Highest marks
- Lowest marks
- Failed subjects
- Grade
- Overall Pass/Fail result

13.4 Main Program

The main.py file connects the different modules and displays the complete performance report for each student.

The project uses functions, loops, conditional statements, lists, dictionaries, and modular programming to implement these features.



14. GRADE CRITERIA

The program assigns grades according to the percentage obtained by the student.

Percentage	Grade
90% and above	A+
80% – 89%	A
70% – 79%	B
60% – 69%	C
50% – 59%	D
40% – 49%	E
Below 40%	F

The grade is calculated automatically by the `grade()` function in the `analysis.py` module.

15. INPUT VALIDATION

Input validation is used to prevent invalid marks from being entered into the program.

The project uses a separate `validation.py` module with the `valid_marks()` function. It checks whether the entered marks are greater than or equal to zero.

If the user enters a negative value, the program displays an error message and asks the user to enter the marks again.

This helps prevent invalid data and improves the reliability of the performance calculations.

16. TESTING

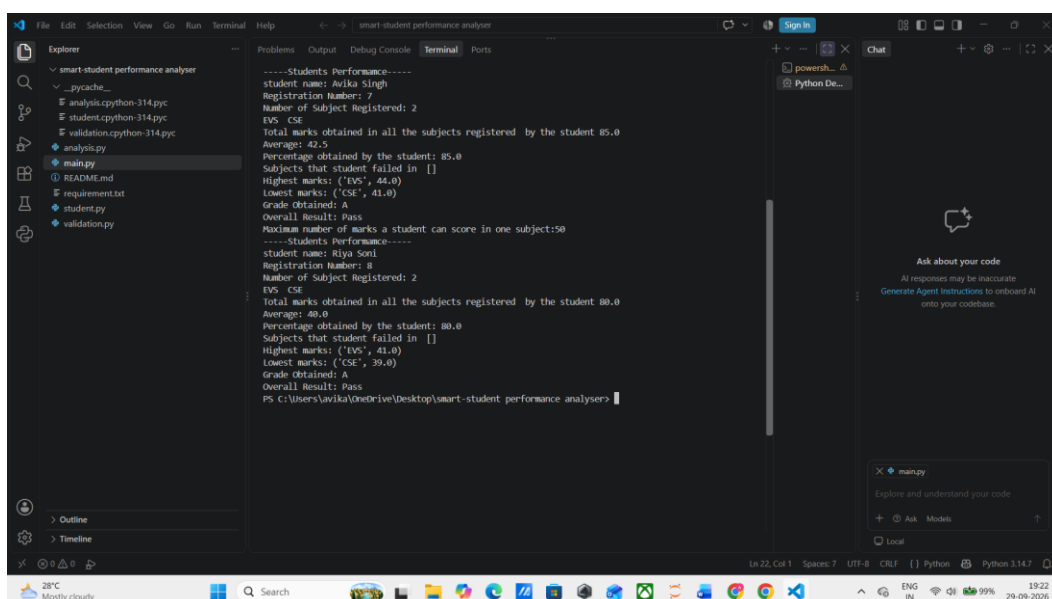
The project was tested with different types of inputs to check whether the calculations and validation were working correctly.

Test Case	Input/Condition	Expected Result
1	Valid marks entered	Marks are accepted and performance is
2	Negative marks entered	Error message is displayed and marks are entered
3	Student with all passing subjects	Overall result is PASS
4	Student with a failed subject	Overall result is FAIL
5	Different percentage values	Correct grade is assigned
6	Multiple students entered	Each student's performance report is displayed

The testing showed that the main functions of the program were working correctly for the tested inputs.

17. SAMPLE OUTPUT

A sample output is shown below:



```
-----Students Performance-----
student name: Aulka Singh
Registration Number: 7
Number of Subject Registered: 2
EVS CSE
Total marks obtained in all the subjects registered by the student 85.0
Average: 42.5
Percentage obtained by the student: 85.0
Subjects that student failed in []
Highest marks: ('EVS', 44.0)
Lowest marks: ('CSE', 41.0)
Grade obtained: A
Overall Result: Pass
Maximum number of marks a student can score in one subject:50

-----Students Performance-----
student name: Riya Soni
Registration Number: 8
Number of Subject Registered: 2
EVS CSE
Total marks obtained in all the subjects registered by the student 80.0
Average: 40.0
Percentage obtained by the student: 80.0
Subjects that student failed in []
Highest marks: ('EVS', 41.0)
Lowest marks: ('CSE', 39.0)
Grade obtained: A
Overall Result: Pass

PS C:\Users\lavika\OneDrive\Desktop\smart-student performance analyser>
```

The output contains the student's basic details along with the calculated academic performance and final result.

18. CHALLENGES FACED

During the development of the Smart Student Performance Analyser, some challenges were faced:

- Understanding how to divide the program into different Python modules.
- Managing marks of different subjects using dictionaries.
- Handling multiple students and displaying their results separately.
- Creating functions for different calculations such as average, percentage, grade, and result.
- Implementing input validation for negative marks.
- Debugging syntax and logical errors while testing the program.

19. LEARNING OUTCOMES

Through the development of the Smart Student Performance Analyser, the following concepts and skills were learned:

- Understanding how to divide a Python project into separate modules.
- Using functions to perform specific tasks.
- Working with lists and dictionaries to store and manage data.
- Using loops and conditional statements for program logic.
- Implementing basic input validation and error handling.
- Understanding how to calculate and analyse student performance using Python.
- Debugging and testing the program with different inputs.
- Understanding the importance of writing organised and maintainable code.

20. FUTURE ENHANCEMENTS

The Smart Student Performance Analyser can be improved further by adding the following features:

Graphical User Interface (GUI): Develop a user-friendly interface using Tkinter.

Data Storage: Store student records permanently using files or a database.

Report Generation: Generate a detailed performance report for each student.

Graphical Representation: Display marks and performance using charts and graphs.

Student Comparison: Add features to compare the performance of multiple students.

Export Option: Allow performance reports to be exported as PDF or other formats.

Improved Validation: Add more checks for invalid or out-of-range inputs.

21. CONCLUSION

The Smart Student Performance Analyser is a Python-based project developed to simplify the process of analysing student academic performance. The program can handle multiple students and calculate important details such as total marks, average, percentage, grade, highest and lowest marks, failed subjects, and overall result.

The project helped in applying basic Python concepts such as functions, loops, conditional statements, lists, dictionaries, input validation, and modular programming in a practical application.

Overall, the project provides a simple foundation that can be further improved with features such as GUI, data storage, report generation, and graphical analysis.

22. REFERENCES

Python Official Documentation — Python programming concepts and built-in functions.

Visual Studio Code Documentation — Code editing and development environment.

Git Documentation — Version control and project management.

GitHub Documentation — Repository management and project documentation.

Course/Faculty Project Guidelines — Requirements and evaluation criteria provided for the project.

Project development was based mainly on Python programming concepts taught in the course.